НИУ ВШЭ НН

ИМИКН

EcoFFT

Архитектура, API и руководство разработчика

Прямое и обратное быстрое преобразование Фурье

Eco ACOM · Linux x86_64 · версия 1.0.0

Автор: Зиновьев Дмитрий Константинович

Группа: 24ТИДИ

Проверяющий: Башев Владимир Николаевич

Нижний Новгород, 2026

Содержание

1. Назначение и возможности

2. Математическая модель

3. Алгоритмы DIT и DIF

4. Произвольная длина: алгоритм Блюстейна

5. Тригонометрия и численные типы без libc

6. Архитектура ACOM и поддержка Eco

7. Интерфейсы и контракты

8. Сервисный и объектный подходы

9. Память: модель и измерения

10. Организация проверок

11. Примеры и сравнение точности

12. Измерение производительности

13. Сборка и практическое использование

14. Устройство проекта и применение Eco.AI

15. Системный промпт модели

16. Ограничения и направления развития

17. Технические источники

1. Назначение и возможности

EcoFFT — библиотека компонентов Eco ACOM для прямого и обратного быстрого преобразования Фурье. Она предназначена для спектрального анализа комплексных сигналов, восстановления сигнала по спектру и блочной фильтрации. Приложение получает вычислительный интерфейс через штатную шину Eco; консольный клиент позволяет обрабатывать собственные данные без написания программы.

Реализованы два взаимозаменяемых компонента с единым контрактом: DIT и DIF. Для длин, не равных степени двойки, каждый использует преобразование Блюстейна. Поддерживаются N от 1 до 65536. Это ограничение делает потребление памяти предсказуемым и позволяет проверять арифметику индексов до выделения буферов.

Вычисления доступны в двух архитектурных формах. Сервис IEcoDSP получает входные и выходные массивы при каждом вызове и не сохраняет сигнал. Объект IEcoFFTPlan владеет копией сигнала и подготовленными таблицами; его можно многократно использовать для одного размера. IEcoDigitalFilter сохраняет спектр ядра циклической свертки. IEcoVector и IEcoMatrix предоставляют собственные комплексные контейнеры.

Возможность	**Публичное средство**
FFT и IFFT для комплексных данных	Семейства fft/ifft, fftf/ifftf, fftl/ifftl
Целочисленный действительный вход	ffti/iffti и fftlong/ifftlong с комплексным результатом
Повторное преобразование одного размера	IEcoFFTPlan: Load, FFT, IFFT, Store
Циклическая свертка	Сервис Filter и объект IEcoDigitalFilter
Векторы и матрицы	CreateVector, CreateMatrix, проверяемые Get/Set
Базовый калькулятор	IEcoCalculatorX: Addition, Subtraction; общий IEcoUnknown с IEcoDSP
Выбор реализации	Отдельные CID, статические и динамические библиотеки DIT/DIF
Пользовательские файлы	CSV-клиент eco_fft, стандартные потоки, сервисный и объектный режимы
Встраивание в приложение	Публичный C-интерфейс и собираемый пример examples/use_component.c

Целевая платформа — Linux x86_64. В Windows сборка и выполнение осуществляются внутри Docker с Linux-контейнерами. Ядро написано на C11, использует собственную комплексную арифметику и выделяет память только через IEcoMemoryAllocator1. Компонент не использует готовые FFT-библиотеки и не вызывает функции libc/libm. Оригинальные заголовки Eco SDK подключаются без изменений, вместе с их косвенными системными объявлениями.

Матрица предоставляет построчное хранение не более 65536 элементов. Двумерное FFT и потоковая обработка с историей задержек не входят в текущий API. Последующие разделы фиксируют математические соглашения, управление состоянием, численные пределы и правила интеграции.

2. Математическая модель

Комплексное число представлено собственной структурой с действительной и мнимой частями. Для двух чисел a и b произведение вычисляется как (a.re·b.re − a.im·b.im, a.re·b.im + a.im·b.re). Заголовок complex.h и готовые комплексные операции не используются.

Прямое преобразование определено для k=0,…,N−1:

$X[k] = \Sigma(n=0\ldots N-1)\ x[n] \cdot \exp(-i\cdot 2\pi\cdot k\cdot n/N)$.

Обратное преобразование определено для n=0,…,N−1:

$x[n] = (1/N) \cdot \Sigma(k=0\ldots N-1)\ X[k] \cdot \exp(+i\cdot 2\pi\cdot k\cdot n/N)$.

Отрицательный знак соответствует FFT, положительный — IFFT. Нормировка 1/N выполняется только в обратном преобразовании. Результат расположен в обычном порядке частот: нулевая частота в элементе 0; fftshift не применяется. Контракт обеспечивает IFFT(FFT(x)) ≈ x с учетом округления. Выбранный коэффициент нормировки зафиксирован в интерфейсе, тестах и эталоне: соглашения разных библиотек могут отличаться [5].

Для импульса x[0]=1 остальные элементы равны нулю, и спектр состоит из единиц. Для постоянного сигнала x[n]=c ненулевым является только X[0]=Nc. Комплексная гармоника exp(i·2π·r·n/N) должна дать пик в позиции r. Эти свойства проверяют знак, порядок частот и коэффициент нормировки независимо от обратного преобразования. Только тест roundtrip был бы недостаточен: одинаковые ошибки FFT и IFFT могли бы взаимно компенсироваться.

Для действительного сигнала спектр в общем случае комплексный. Поэтому целочисленные варианты принимают int или long, а возвращают EcoComplexD. Округлять спектр до целых чисел нельзя без потери результата. В iffti и ifftlong целые входы трактуются как действительные значения спектра; восстановленный сигнал также комплексный.

3. Алгоритмы DIT и DIF

Основа radix-2 — разложение преобразования размера N на два преобразования N/2 и стадию объединения. Работа Cooley–Tukey служит исходным математическим ориентиром [3]. Реализация выполнена самостоятельно, готовый код FFT из SDK или внешней библиотеки не включается.

3.1. Decimation in Time

В DIT сначала выполняется битовая перестановка индексов. Затем длина блока последовательно увеличивается: 2, 4, 8, …, N. Для каждой пары берутся u=a[k] и t=W·a[k+L/2], после чего записываются u+t и u−t. Здесь W=exp(−i·2π·j/L). Для IFFT знак мнимой части W меняется на противоположный.

```
bit_reverse(data)
for L = 2, 4, ..., N:
   for each block of length L:
      for j = 0, ..., L/2 - 1:
         u = data[block+j]
         t = W[j*N/L] * data[block+j+L/2]
         data[block+j] = u+t
         data[block+j+L/2] = u-t
if inverse: divide every element by N
```

Перестановка до вычислений обеспечивает естественный порядок результата. Ядро итеративное и не использует рекурсивный стек. Поворотные множители вычисляются при создании численного плана и затем только читаются.

3.2. Decimation in Frequency

В DIF длина блока уменьшается: N, N/2, …, 2. Сначала вычисляются сумма u+v и разность u−v, затем разность умножается на W. Выход промежуточных стадий находится в битово переставленном порядке, поэтому финальная перестановка выполняется после всех бабочек.

```
for L = N, N/2, ..., 2:
   for each block of length L:
      for j = 0, ..., L/2 - 1:
         u = data[block+j]
         v = data[block+j+L/2]
         data[block+j] = u+v
         data[block+j+L/2] = (u-v) * W[j*N/L]
bit_reverse(data)
if inverse: divide every element by N
```

DIT и DIF реализованы двумя разными циклами бабочек. Это не два названия одного вызова. Каждый алгоритм опубликован отдельной фабрикой и отдельной библиотекой с собственным CID. Код жизненного цикла, контейнеров и универсальных преобразований типов общий; это устраняет дублирование архитектурной обвязки, сохраняя выбор алгоритма в клиенте.

3.3. Асимптотика

Для $N=2^p$ число стадий равно $\log_2 N$. На каждой стадии выполняется N/2 комплексных бабочек. Следовательно, время преобразования O(N log N), битовой перестановки O(N), подготовки таблиц O(N). Внутренний буфер и таблицы занимают O(N) памяти; рекурсивная глубина отсутствует. Прямой ДПФ требует $O(N^2)$ комплексных операций и используется только в независимом тестовом эталоне.

4. Произвольная длина: алгоритм Блюстейна

Простой radix-2 не определяет вычисление при N=7 или N=127. Вместо скрытого дополнения исходного сигнала нулями, которое изменило бы частотную сетку, реализовано преобразование Блюстейна [4]. Оно вычисляет именно N-точечное ДПФ, используя вспомогательную свертку большей длины.

Из тождества $2kn=k^2+n^2-(k-n)^2$ следует:

$c[n] = \exp(-i\cdot\pi\cdot n^2/N)$, $a[n]=x[n]c[n]$, $b[t]=\exp(+i\cdot\pi\cdot t^2/N)$.

$X[k] = c[k] \cdot \Sigma(n=0\ldots N-1)\ a[n] \cdot b[k-n]$.

Выбирается M — минимальная степень двойки не меньше 2N−1. Положительные индексы b записываются в начало массива, отрицательные — в позиции M−t. Остальные элементы нулевые. Свертка получается как IFFT_M(FFT_M(a)·FFT_M(b)); первые N результатов умножаются на c[k]. Внутренняя IFFT делится на M, а внешняя обратная операция использует сопряжение и отдельный делитель N. Смешивать эти две нормировки нельзя.

В реализации сначала вычисляется целочисленный остаток $n^2 \bmod 2N$, затем угол. При $N \le 65536$ квадрат максимального индекса помещается в uint32_t. Угол не растет пропорционально N^2, что уменьшает ошибку приведения диапазона. В плане сохраняются chirp, FFT(b), таблица поворотных множителей и рабочий буфер. Повторное применение подготовленного численного плана не выделяет память.

Поскольку M<4N, сохраняются O(N log N) времени и O(N) памяти. Однако константы выше, чем для прямого radix-2. Поэтому соседние длины 65535 и 65536 могут иметь существенно разное время и объем памяти. В обоих компонентах используется одна схема Блюстейна, но ее внутренние FFT выполняются выбранным DIT либо DIF.

5. Тригонометрия и численные типы без libc

В компоненте отсутствуют sin, cos, pow, sqrt, malloc, memcpy и другие функции стандартной библиотеки. Численное ядро получает ограниченный угол, приводит его по четвертям к интервалу $[-\pi/4, \pi/4]$, отдельно вычисляет ряды синуса и косинуса и восстанавливает знаки и перестановку функций.

Начальные члены $s_0=x$ и $c_0=1$. Для $j=1,\ldots,12$ используются рекуррентные формулы членов ряда:

$s_j = s_{j-1} \cdot (-x^2)/((2j)(2j+1))$; $c_j = c_{j-1} \cdot (-x^2)/((2j-1)(2j))$.

Суммируются начальные значения и 12 обновлений. В этой области остаточный член имеет порядок меньше $3 \cdot 10^{-29}$; фактическая точность ограничена округлением long double, а не только остатком ряда. Каждый поворотный множитель рассчитывается независимо. Длинная цепочка умножений W←W·Wstep, накапливающая ошибку фазы, не применяется.

Внутренняя арифметика всех вариантов использует long double. На проверенной платформе это 64 значащих двоичных разряда, 16 байт хранения скаляра и 32 байта EcoComplexL. float и double расширяются перед расчетом, затем результат сужается один раз. Это архитектурное решение повышает точность промежуточных операций, но означает, что float-вариант не является оптимизированным вычислением в одинарной точности. Его скорость нельзя напрямую сопоставлять с SIMD FFT для float.

Для float ограничен модуль каждой входной части величиной 10^{30}; для остальных типов — 10^{100}. NaN и бесконечности отвергаются упорядоченными сравнениями. Векторы и внутреннее состояние плана также ограничены величиной 10^{100}. План проверяет результат

до замены старого сигнала. Очень большой допустимый вход сервисного FFT может дать результат за пределом входного диапазона следующего IFFT; это явно ограничивает область обещания roundtrip. Для произвольных амплитуд преобразование масштабировать автоматически не пытается.

6. Архитектура ACOM и поддержка Eco

6.1. Получение компонента через среду

Точку входа main предоставляет оригинальная библиотека Eco.System1, вызывающая int16_t EcoMain(IEcoUnknown*). Клиент получает IEcoSystem1, затем IEcoInterfaceBus1 через QueryInterface. В поставленном StaticRelease runtime системные компоненты, включая менеджер памяти, уже подготовлены. Для FFT клиент вызывает QueryComponent с CID выбранного алгоритма и IID_IEcoDSP. Шина сама находит файл <CID>.so, получает стандартную фабрику и создает экземпляр; клиент не вызывает фабрику и не содержит код FFT. Минимальный пример работает из каталога компонентов. CLI временно выбирает этот каталог на время QueryComponent и восстанавливает рабочий каталог до чтения CSV. По умолчанию используются библиотеки рядом с исполняемым файлом; --dynamic задает другой каталог.

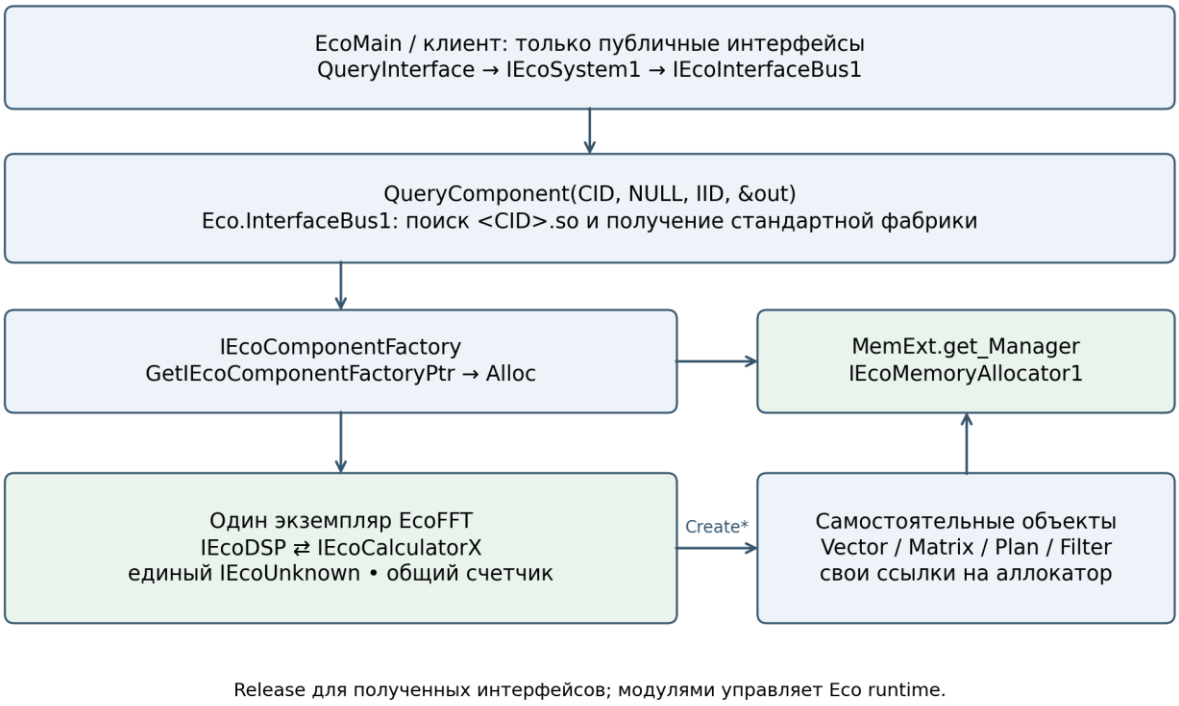


Рисунок 1 — получение экземпляра через Eco и идентичность интерфейсов.

6.2. Интерфейсы и общая идентичность

Первые три метода каждой таблицы — QueryInterface, AddRef и Release. Один экземпляр сервиса поддерживает IEcoDSP и исходный IEcoCalculatorX: QueryInterface позволяет переходить между ними в обе стороны. Оба возвращают один канонический

IEcoUnknown и разделяют счетчик ссылок. Указатель IEcoCalculatorX расположен со смещением внутри объекта; его методы сначала восстанавливают адрес владельца. Успешный запрос добавляет одну принадлежащую вызывающему ссылку. При последнем Release освобождаются объект и удерживаемая ссылка на аллокатор. Набор поддерживаемых IID остается постоянным на протяжении жизни экземпляра.

Контракт IEcoCalculatorX сохранен: IID 93221116-2248-4742-AE06-82819447843D, Addition(int16_t,int16_t) возвращает int32_t, Subtraction(int16_t,int16_t) — int16_t. Выход разности за 16-битный диапазон обрабатывается циклически, без неопределенного переполнения знакового типа. Методы Add и Subtract прежнего IEcoDSP также сохранены. Неподдерживаемый IID возвращает ошибку с *out=NULL. Агрегация не реализована: ненулевой outer отклоняется фабрикой с ERR_ECO_NOAGGREGATION. Векторы, матрицы, планы и фильтры являются отдельными объектами со своей идентичностью и счетчиком ссылок.

Сущность	**Идентификатор**
CID DIT	C345FB78-7071-4410-9821-A6EF25310070
CID DIF	C345FB78-7071-4410-9821-A6EF25310071
IID IEcoDSP	C345FB78-7071-4410-9821-A6EF25310001
IID IEcoVector	C345FB78-7071-4410-9821-A6EF25310002
IID IEcoMatrix	C345FB78-7071-4410-9821-A6EF25310003
IID IEcoFFTPlan	C345FB78-7071-4410-9821-A6EF25310004
IID IEcoDigitalFilter	C345FB78-7071-4410-9821-A6EF25310005
IID IEcoCalculatorX	93221116-2248-4742-AE06-82819447843D

6.3. Фабрика, ABI и зависимости

Фабрика реализует официальный IEcoComponentFactory. В .so при ECO_DLL экспортируется только GetIEcoComponentFactoryPtr; в .a при ECO_LIB предоставляется указатель GetIEcoComponentFactoryPtr_<32-значный CID>. Декларации статических фабрик находятся в include/IdEcoFFT.h. Статическая цель eco_fft_static регистрирует эти указатели через RegisterComponent — штатный механизм ACOM. Основное приложение и интеграционные примеры используют динамическое обнаружение. Модулями и зарегистрированными фабриками управляет runtime; клиент освобождает полученные интерфейсы и не вызывает dlclose.

Компонент и клиент компилируются с неизмененными официальными заголовками, включая depend.h, IEcoBase1.h и IEcoSystem1.h. Используются определения UGUID и ECOCALLMETHOD из SDK; для этой платформы sizeof(UGUID)=18. Проверка SHA256 подтверждает происхождение всех сохраненных SDK-файлов. Системные объявления, косвенно подключаемые depend.h, не отключаются и не заменяются собственными typedef. Запрет библиотечных функций проверяется отдельно по вызовам в авторском коде и импортам готовых .so.

Alloc получает системный интерфейс, шину и IEcoInterfaceBus1MemExt. Аллокатор запрашивается через QueryComponent по CID, возвращенному get_Manager; только при отсутствии расширения используется стандартный CID_EcoMemoryManager1. Компонент не регистрирует зависимости и не подменяет выбранный средой менеджер. Все временные интерфейсные ссылки освобождаются, а созданный сервис удерживает аллокатор. Alloc возвращает полностью инициализированный stateless-сервис; Init не требует дополнительного состояния. Реализация написана на C11; стандартные библиотеки используются только в клиентском вводе/выводе и независимых проверочных эталонах.

7. Интерфейсы и контракты

Полный публичный контракт находится в include/IEcoDSP.h. Во всех методах первый параметр me обозначает экземпляр интерфейса. В таблице input — входной массив, output — буфер результата длины N, out — адрес указателя на создаваемый объект. Внешними массивами владеет клиент; компонент не освобождает их и не сохраняет после сервисного вызова.

Методы	**Вход → выход**	**Назначение**
fft / ifft	EcoComplexD[N] → EcoComplexD[N]	Двойная точность интерфейса
fftf / ifftf	EcoComplexF[N] → EcoComplexF[N]	Одинарная точность интерфейса
fftl / ifftl	EcoComplexL[N] → EcoComplexL[N]	Расширенная точность
ffti / iffti	int[N] → EcoComplexD[N]	Действительные целочисленные данные
fftlong / ifftlong	long[N] → EcoComplexD[N]	Целые типа long
CreateVector	N → IEcoVector	Собственный комплексный вектор
CreateMatrix	rows, cols → IEcoMatrix	Плотная матрица в построчном порядке
CreatePlan	N → IEcoFFTPlan	Объект с таблицами и внутренним сигналом
CreateFilter	IEcoVector → IEcoDigitalFilter	Снимок ядра циклической свертки
Filter	signal, kernel, output	Сервисная свертка без сохранения ядра
Add / Subtract	long double, long double → long double	Базовые арифметические операции
Algorithm / MemoryBytes	экземпляр → uint32_t	Выбранный алгоритм / объем объекта

Пример вызова метода в стиле ACOM:

```
EcoComplexD input[4] = {{1,0},{2,0},{3,0},{4,0}};
EcoComplexD spectrum[4];
int16_t rc = dsp->pVTbl->fft(dsp, 4, input, spectrum);
/* rc == 0: spectrum = {10, -2+2i, -2, -2-2i}. */
```

Точное совпадение input и output поддерживается. Частичное перекрытие не является обещанным контрактом публичного интерфейса, даже если внутренний scratch делает некоторые такие случаи работоспособными. При ошибке аргумента или выделения памяти сервис не записывает результат. У произвольного внешнего IEcoVector метод Set может завершиться ошибкой после нескольких успешных записей: такую ошибку Store/Apply передает клиенту; полного отката стороннего контейнера контракт не обещает.

Код	**Значение**
0, ECO_OK	Успех
−1, ECO_E_ARGUMENT	Нулевой или некорректный аргумент
−2, ECO_E_MEMORY	Отказ выделения памяти
−3, ECO_E_RANGE	Длина, индекс, размерность или значение за пределами контракта
−4, ECO_E_STATE	План не загружен либо внутренний план занят
−5, ECO_E_INTERFACE	Интерфейс не поддерживается

Интерфейсы данных имеют Get/Set с проверкой индексов и MemoryBytes. Нулевые размеры запрещены. Произведение rows·cols проверяется делением до умножения, что исключает переполнение. Объекты однопоточные: совместное использование одного плана или счетчика ссылок требует внешней синхронизации; независимые экземпляры имеют собственные рабочие буферы.

8. Сервисный и объектный подходы

8.1. Сервис без состояния

IEcoDSP хранит только таблицу методов, счетчик ссылок, выбранный алгоритм и ссылку на аллокатор. Сигнал, преобразованные данные и коэффициенты между вызовами не сохраняются. Каждый fft/ifft создает численный план и буфер преобразования типов, рассчитывает результат и освобождает временную память. Сервис является stateless относительно вычислительных данных; наличие инфраструктурного счетчика ссылок не делает его состоянием сигнала.

Преимущество сервиса — короткий вызов и независимость от порядка предшествующих вычислений. Он удобен для единичного преобразования или постоянно меняющегося N. Цена — повторное выделение памяти, вычисление таблиц и, для Блюстейна, подготовка FFT ядра свертки при каждом вызове.

8.2. Объект IEcoFFTPlan

CreatePlan фиксирует N и выбранный алгоритм. Объект содержит numerical kernel, собственный signal[N], staging[N] и признак loaded. Load читает внешний вектор в staging, проверяет все значения и только после успеха заменяет внутренний сигнал. Изменение исходного вектора после Load не изменяет объект.

```
/* vec and result were created via CreateVector, plan via CreatePlan. */
rc = plan->pVTbl->Load(plan, vec);
if (rc == 0) rc = plan->pVTbl->FFT(plan);
if (rc == 0) rc = plan->pVTbl->Store(plan, result);
if (rc == 0) rc = plan->pVTbl->IFFT(plan);
/* Release each successfully created interface exactly once. */
```

FFT/IFFT не принимают массив снаружи: они преобразуют сохраненные данные объекта. Store копирует текущее состояние в предоставленный вектор. До Load методы преобразования и Store возвращают ECO_E_STATE. Ошибка Load или преобразования не заменяет предыдущее корректное состояние. Два плана, созданные одним сервисом, не разделяют сигналы и рабочие массивы.

План полезен при многократной работе с одним размером: подготовленные коэффициенты сохраняются, вызовы не выделяют новую память. Но клиент должен управлять последовательностью состояний и временем жизни объекта. Дополнительные Get/Set и копирования могут уменьшить выигрыш от кэша; это отдельно измерено.

8.3. Объект IEcoDigitalFilter

При CreateFilter вектор коэффициентов копируется и преобразуется в спектр H. Apply(signal,result) вычисляет Y=FFT(signal)·H и затем IFFT(Y). Повторный Apply не читает исходное ядро и не пересчитывает его спектр. Изменение коэффициентов в исходном векторе после создания не меняет объект. Сервисный Filter создает аналогичный временный объект на время одного вызова.

Операция представляет N-точечную циклическую свертку: $y[n]=\Sigma\, h[j]\cdot x[(n-j) \bmod N]$. Пример $h=\{0.5,0.5,0,\ldots\}$ дает $y[n]=(x[n]+x[(n-1) \bmod N])/2$. Это блочный FIR-пример с неизменяемыми коэффициентами; он не моделирует потоковый IIR, историю задержек или линейную свертку без циклического перехода.

Перед обратным преобразованием проверяется диапазон спектрального произведения, а перед записью наружу — весь результат. Неверный вход не разрушает кэш. Демонстрация и UnitTest проверяют повторные вызовы, одинаковые результаты SOA/OOA, совпадение входного и выходного векторов и неизменность сохраненных коэффициентов.

9. Память: модель и измерения

MemoryBytes сообщает сумму запрошенных байт объекта и всех принадлежащих ему буферов. Служебные заголовки аллокатора, память кода, системной шины и клиентские массивы в эту величину не входят. Корректность метода проверяется независимым счетчиком запросов, который делегирует реальное выделение официальному Eco-аллокатору.

На проверенном ABI sizeof(EcoComplexL)=32. Сервис занимает 40 байт, включая два указателя интерфейсов; вектор — 32+32N, матрица — 40+32·rows·cols. Численное ядро K(N) при $N=2^p \geq 2$ занимает 64+48N байт: служебные поля, N/2 коэффициентов и N элементов scratch. План добавляет 56+64N байт на объект и два массива. Итого план: 120+112N. Для N=1 размер таблицы имеет отдельный минимум в один элемент.

Для Блюстейна K(N)=64+80M+32N, где M=nextpow2(2N−1). План занимает 120+80M+96N. Фильтр отличается от плана размером служебной структуры: 48 вместо 56 байт, при тех же двух собственных N-массивах и численном ядре.

N	Сервис, байт	Вектор, байт	План, байт	Фильтр, байт
16	40	544	1912	1904
127	40	4096	32792	32784
1024	40	32800	114808	114800
65535	40	2097152	16777240	16777232
65536	40	2097184	7340152	7340144

У stateless-сервиса малая постоянная память, но временная память не равна нулю. Пик на один расчет равен 40+32N+K(N) без внешних входного и выходного массивов. План сохраняет все свои массивы между вызовами. Если сравнивать полный клиентский сценарий, дополнительно учитываются два внешних массива у SOA или два объекта-вектора у OOA; 40 байт сервиса и полный план не являются одинаковыми по смыслу показателями пикового потребления.

10. Организация проверок

Основной тестовый клиент получает компоненты через штатную Eco InterfaceBus и динамические библиотеки, не связываясь с реализацией FFT. Отдельно проверяется статическая цель eco_fft_static со стандартными фабричными указателями. Внутренние проверки отказов выделения вынесены в eco_fft_faults: этот тест явно использует код реализации и tracking allocator. Его результаты не выдаются за проверку внешнего подключения. Под AddressSanitizer и UndefinedBehaviorSanitizer инструментируются клиент и авторские компоненты, включая обе .so; предварительно собранные библиотеки SDK не инструментируются.

Запуск	Проверок	Ошибок	Итог	Журнал в build/results/
Динамический FFT/API	3936	0	PASS	tests.log
Статический FFT/API	3936	0	PASS	static-tests.log
Отказы и время жизни	5093	0	PASS	faults.log
Динамический FFT/API + ASan/UBSan	3936	0	PASS	sanitizer.log
Отказы + ASan/UBSan	5093	0	PASS	faults-sanitizer.log
ACOM-контракт (Release / ASan)	134 / 134	0 / 0	PASS	acom-contract*.log
Развертывание (Release / ASan)	19 / 19	0 / 0	PASS	acom-integration.json, acom-sanitizer.json

Математический клиент выполняет 3936 проверок на запуск. Отдельный тест отказов и времени жизни выполняет 5093 проверки и обрабатывает 722 вызова выделения с инъекцией отказов. Эти числа относятся к разным исполняемым программам; повторные запуски под санитайзерами не увеличивают число уникальных сценариев. Дополнительные проверки ACOM и развертывания приведены в подразделе 10.5.

10.1. Независимые математические проверки

Эталон точности — прямое ДПФ $O(N^2)$, написанное в клиенте с sinl/cosl из math.h. В нем нет вызовов проверяемого ядра. Фаза вычисляется непосредственно по паре индексов k,n; для уменьшения погрешности аргумент приводится целочисленным остатком modulo N. Сравниваются и прямое, и обратное преобразования, а не только их композиция.

Для каждого алгоритма берутся N={1,2,3,5,7,8,16,31,64,127,256,257}. Для каждого размера используются пять сигналов: импульс, константа, действительная гармоника, комплексная гармоника и псевдослучайная комплексная последовательность с фиксированным начальным состоянием. Проверяются пять типов и два направления: всего 600 строк измерений для DIT и 600 для DIF.

Эталон получает именно представимые входные значения выбранного типа. Например, float сначала округляется до float, затем расширяется для ДПФ; это отделяет ошибку алгоритма от исходного квантования данных. Для int используется целочисленный сигнал, для long — его увеличенная версия. Величины ошибок long поэтому нельзя интерпретировать как точность float на единичном диапазоне.

Для N=4096,65535,65536 проверяются roundtrip и равенство Парсеваля. Аналитические тесты отдельно проверяют положение пиков, знак мнимой части и нормировку. In-place запуск сравнивается с отдельным выходным массивом.

10.2. Контракт, жизненный цикл и ошибки

Проверяются нулевые указатели, N=0 и N>65536, NaN/Inf, выход за границы вектора/матрицы, неизвестные IID, выходные указатели при ошибке, состояние плана до Load, несовпадение размеров, владение ссылками и неизменность внешних данных после их копирования. В тесте внешний вектор специально возвращает ошибку Get: старое состояние плана должно остаться доступным.

Tracking allocator фиксирует каждый запрос и освобождение, суммарный объем и число ссылок. Для конструкторов сервиса, вектора, матрицы, плана и фильтра последовательно моделируется отказ на каждой точке выделения до первого успешного полного создания. Аналогично проверяются временные выделения сервисных FFT и Filter. После отказа не должно оставаться частично созданных объектов или потерянных ссылок. Повторные операции готового плана и фильтра не должны увеличивать число Alloc.

10.3. Запрет libc

Сборка компонента использует -ffreestanding, -fno-builtin и -fno-tree-loop-distribute-patterns, чтобы компилятор не подменял написанный цикл вызовом memcpy/memset.

Динамические библиотеки линкуются с -nostdlib, -nodefaultlibs и -Wl,-z,defs. Проверяются исходники, фактические зависимости препроцессора и все неопределенные символы, а не только короткий список известных функций.

Компонент	Неопределенных символов	DT_NEEDED	Итог
0070	0	нет	PASS
0071	0	нет	PASS

Аудит охватил 9 авторских файлов .c/.h и проверил SHA256 16 сохраненных SDK-файлов. В каждую единицу трансляции компонента включен оригинальный depend.h; косвенные системные объявления допускаются. Ни у одной Release-библиотеки нет неопределенных символов или DT_NEEDED; единственный динамический экспорт — GetIEcoComponentFactoryPtr. Это утверждение относится к авторским компонентам, а не к внутренней реализации поставляемого runtime Eco. Результат, список включенных заголовков и SHA256 библиотек записываются в build/results/audit.json.

10.4. Внешний интерфейс и пример интеграции

Динамический CSV-клиент прошел 105 проверок в Release и 105 под ASan/UBSan. Проверены стандартный поиск рядом с исполняемым файлом и явный каталог --dynamic. Отдельный статический клиент прошел 68 проверок. Результаты сравниваются с независимым ДПФ на Python для DIT/DIF, сервисного и объектного API, прямого и обратного преобразований. Проверены N=1,4,7, композиция FFT/IFFT и максимальная длина 65536 на импульсном сигнале.

Отдельно проверены чистый CSV в stdout, стандартный ввод, файлы с пробелами в пути, BOM/CRLF, комментарии, пустой сигнал, неверные поля, NaN/Inf, превышение числового диапазона, слишком длинная строка, N>65536, отсутствие входа и библиотеки, ошибки открытия и записи выхода. Во всех сценариях неверного входа ранее существовавший выходной файл сохранил содержимое. Пример examples/use_component.c собран и выполнил FFT/IFFT с проверкой известного результата.

Журналы CLI сохраняются в build/results/cli-checks.json, cli-sanitizer.json и cli-static.json. Числа обозначают проверки сценариев клиента и не складываются с числом проверок математического компонента как мера покрытия кода.

10.5. Проверка компонентной среды и заменяемости

Самостоятельное приложение examples/check_acom.c связывается только с официальным runtime и публичными заголовками. Оно выполнило 134 проверки: рефлексивность, симметричность и транзитивность QueryInterface, постоянство набора IID, канонический IEcoUnknown для IEcoDSP/IEcoCalculatorX, баланс ссылок, независимость экземпляров и жизнь дочерних объектов после освобождения сервиса. Проверены прямое получение IEcoCalculatorX через QueryComponent, отказ агрегации, неизвестный IID и

соблюдение выбранного средой менеджера памяти. При недоступном менеджере создание действительно завершается ошибкой; после восстановления настройки снова работает.

Скрипт scripts/check_acom.py выполнил 19 проверок развертывания в отдельном временном каталоге. Анализ символов подтвердил отсутствие реализации FFT и фабрик EcoFFT в клиентских бинарниках и отсутствие dlopen/dlsym/dlclose в их авторских объектных файлах. Без .so минимальный клиент не работает. После развертывания библиотеки вычисления проходят. Замена поставщика под тем же именем CID с DIT на совместимую реализацию DIF меняет Algorithm(), сохраняя SHA256 клиентского бинарника. Проверены отказ при поврежденной .so и путь с пробелами. Трасса LD_DEBUG показывает загрузку нужного файла и разрешение GetIEcoComponentFactoryPtr средствами runtime.

Обе серии повторены с ASan/UBSan без ошибок. Протоколы: build/results/acom-contract.log, acom-integration.json, acom-loader.log; варианты с санитайзерами — acom-contract-sanitizer.log, acom-sanitizer.json и acom-loader-sanitizer.log. Проверка отсутствующей или замененной библиотеки выполняется в новом процессе, чтобы исключить уже загруженный модуль из предыдущего сценария.

11. Примеры и сравнение точности

11.1. Сравнение с прямым ДПФ на math.h

Метрика абсолютной ошибки: $E_{abs}=\max_k(\max(|\mathrm{Re}(Y_k-Yref_k)|, |\mathrm{Im}(Y_k-Yref_k)|))$. Масштабированная ошибка делит эту величину для каждого k на $\max(1, |\mathrm{Re}(Yref_k)|, |\mathrm{Im}(Yref_k)|)$. Она не является чистой относительной ошибкой возле нулевого спектрального коэффициента. Roundtrip сравнивает исходный представимый сигнал с IFFT(FFT(x)).

Количество десятичных знаков d в таблице выбрано консервативно по условию $E_{abs}<0.5\cdot10^{-d}$. Это эмпирическая оценка для испытанного набора и его амплитуд; она не является доказательством такой же точности на всех допустимых входах. В таблице показан максимум по обоим алгоритмам, всем размерам и сигналам; FFT и IFFT разделены.

Тип	Направление	Eabs	Escaled	Знаков d	Roundtrip
float	FFT	5.612e-07	5.818e-08	5	2.553e-08
float	IFFT	1.855e-08	1.855e-08	7	—
double	FFT	2.248e-15	1.084e-16	14	4.982e-17
double	IFFT	3.415e-17	3.415e-17	16	—
long double	FFT	5.551e-17	3.781e-17	15	1.735e-18
long double	IFFT	3.271e-19	3.271e-19	18	—
int	FFT	7.532e-13	4.072e-15	11	6.863e-15
int	IFFT	3.431e-15	1.022e-16	14	—
long	FFT	4.329e-06	3.293e-08	5	5.402e-08
long	IFFT	2.712e-08	3.020e-10	7	—

Для long double максимальная абсолютная ошибка прямого преобразования равна 5.551e-17, обратного — 3.271e-19. Для double ошибка прямого преобразования составляет 2.248e-15; для float — 5.612e-07. Различие объясняется прежде всего форматом выдаваемого результата. Даже расширенный эталон имеет конечную точность; таблица измеряет расхождение двух вычислительных реализаций, а не ошибку относительно бесконечно точной арифметики. Проверки больших N отдельно подтверждают восстановление и энергетическое соотношение; их нельзя смешивать с приведенной таблицей прямого сравнения с ДПФ.

Следующий пример использует x={1,2,3,4,5,6,7,8}. В нем X[0]=36, X[4]=−4, остальные коэффициенты образуют комплексно-сопряженные пары. Для каждой строки сравнение с math.h ДПФ сохранено в build/results/demo-dit.csv; аналогичная таблица имеется для DIF.

k	**x[k]**	**Re FFT**	**Im FFT**	**Re IFFT**	**Re эталона**
0	1	36.000000000	0.000000000	1.000000000	36.000000000
1	2	-4.000000000	9.656854249	2.000000000	-4.000000000
2	3	-4.000000000	4.000000000	3.000000000	-4.000000000
3	4	-4.000000000	1.656854249	4.000000000	-4.000000000
4	5	-4.000000000	0.000000000	5.000000000	-4.000000000
5	6	-4.000000000	-1.656854249	6.000000000	-4.000000000
6	7	-4.000000000	-4.000000000	7.000000000	-4.000000000
7	8	-4.000000000	-9.656854249	8.000000000	-4.000000000

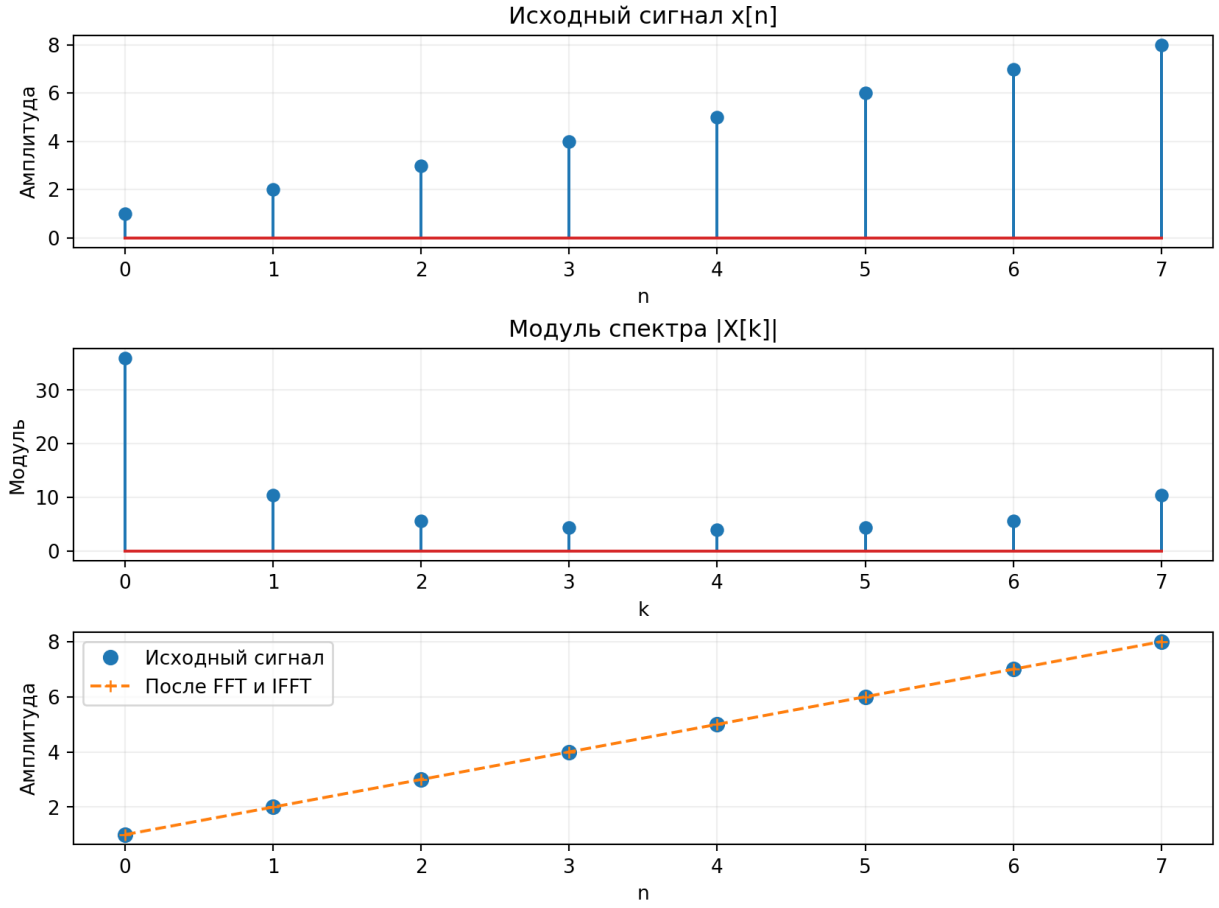


Рисунок 2 — исходные отсчеты, модуль FFT и восстановление IFFT для примера N=8.

В консольной демонстрации все коэффициенты выведены с 12 знаками после запятой. Кроме сравнения с ДПФ клиент проверяет точное совпадение результатов SOA и OOA для этого примера. Восстановленные действительные значения совпадают с исходными до показанных 9 знаков; исходные CSV сохраняют до 18 значащих цифр.

11.2. Сравнение с FFTW

Дополнительный независимый ориентир — библиотека FFTW, фактическая версия fftw-3.3.10. Клиент client/compare_fftw.c вызывает fftwl_plan_dft_1d и fftwl_execute через публичный API [5–7]. Обе стороны получают одинаковые комплексные значения long double. FFTW_BACKWARD возвращает ненормированный результат; клиент делит обе его части на N. Сравнение выполняется непосредственно для FFT и IFFT, а не только по восстановлению сигнала.

Проверены 18 длин: 1, 2, 3, 5, 7, 8, 16, 31, 64, 127, 256, 257, 1023, 1024, 4096, 16384, 65535 и 65536. Для каждой длины используются импульс, константа, действительная косинусоида, комплексная гармоника и детерминированный псевдослучайный сигнал. DIT/DIF, FFT/IFFT и SOA/OOA дают 720 сравнений; все завершились успешно. Порог приемки Escaled<$2\cdot10^{-14}$; в таблице приведены фактические максимумы, а не значения допуска. Те же 720 сравнений отдельно прошли под ASan/UBSan, включая путь создания и освобождения планов FFTW. Инструментированы авторский код и клиент; готовые двоичные библиотеки Eco runtime и FFTW не инструментированы.

Алгоритм	API	Операция	Eabs	Escaled	Знаков d
DIF	OOA	FFT	1.421e-14	5.432e-15	13
DIF	OOA	IFFT	2.711e-19	2.711e-19	18
DIF	SOA	FFT	1.421e-14	5.432e-15	13
DIF	SOA	IFFT	2.711e-19	2.711e-19	18
DIT	OOA	FFT	1.421e-14	5.911e-15	13
DIT	OOA	IFFT	3.116e-19	3.116e-19	18
DIT	SOA	FFT	1.421e-14	5.911e-15	13
DIT	SOA	IFFT	3.116e-19	3.116e-19	18

Количество знаков d вычислено по тому же условию $Eabs<0.5\cdot10^{-d}$ и ограничено 18 для данной платформы. Оно описывает только испытанные сигналы. Это расхождение двух реализаций конечной точности, а не доказательство ошибки относительно точного математического значения. Все 720 строк сохраняются в build/results/fftw-accuracy.csv. Пять публичных числовых типов дополнительно проверяются прямым ДПФ из предыдущего подраздела; сравнение FFTW здесь выполнено для long double.

12. Измерение производительности

12.1. Методика и среда

Для сравнения со стандартным аналогом используется FFTW [5–7]. Она подключается только к отдельному исполняемому файлу eco_fft_reference и не входит в компоненты, их публичный API или основной клиент eco_fft. Прямой ДПФ и рекурсивный FFT на sinl/cosl сохранены в обычном режиме --benchmark как дополнительные алгоритмические ориентиры. Приведенные далее таблицы получены реальным запуском FFTW и EcoFFT в одном процессе.

Параметр	Фактическое значение
UTC выполнения	2026-10-15T15:50:15.884755+00:00
CPU	AMD Ryzen 5 4600H with Radeon Graphics
Платформа	Linux-6.18.54.1-microsoft-standard-WSL2-x86_64-with-glibc2.41
Компилятор	cc (Debian 14.2.0-19) 14.2.0
CMake	cmake version 3.31.6
Оптимизация	Release (-O3 -DNDEBUG), без fast-math
Тип арифметики	long double; LDBL_MANT_DIG=64, sizeof(long double)=16
FFTW	fftw-3.3.10; FFTW_ESTIMATE, один поток

Используются CLOCK_MONOTONIC, прогрев и семь измеряемых пакетов; результат — медиана микросекунд на операцию. Для каждой метрики число повторов удваивается до длительности прогревочного пакета не менее 10 мс либо до 65536 повторов. В измеряемых пакетах сохраняется выбранное число повторов. Оно записано отдельными колонками CSV. Так короткие операции не сравниваются по одному вызову таймера. Результат потребляется

вне измеряемого интервала; ошибка API, нечисловой результат или сбой записи дают ненулевой код завершения.

Полный вызов SOA включает выделение временной памяти, подготовку коэффициентов, FFT/IFFT и освобождение. Полный вызов FFTW включает выделение двух выровненных буферов, создание плана, копирование входа, выполнение, нормировку IFFT и копирование выхода, уничтожение плана и освобождение буферов. Для повторного использования EcoFFT измеряет Load+FFT/IFFT+Store с уже созданными векторами и планом; FFTW измеряет копирование входа, выполнение готового плана, нормировку IFFT и копирование результата.

FFTW работает в long double через fftwl_*, в одном потоке и с FFTW_ESTIMATE. Внешний файл wisdom не загружается; внутрипроцессный кэш планировщика прогрет, поэтому полный вызов не означает холодный старт процесса. FFTW_MEASURE и другие режимы поиска плана не исследованы. Создание/удаление планов записано отдельно в CSV: eco_setup_us включает внутренние буферы и состояние EcoFFT, fftw_setup_us — один направленный план FFTW с готовыми внешними буферами. Эти две колонки описывают разный объем объектов и не равны полному времени первого преобразования.

12.2. Результаты FFT и IFFT

FFT: медиана времени, мкс. «Полный» соответствует полному вызову; «повтор» — готовому плану.

Алгоритм	N	Eco SOA	FFTW полный	Eco OOA	FFTW повтор
DIT	16	1.174	2.023	1.149	0.363
DIT	64	5.496	5.421	5.341	2.110
DIT	127	63.397	55.682	37.509	27.239
DIT	256	24.179	15.135	24.616	11.672
DIT	1023	682.770	120.631	375.434	106.418
DIT	1024	112.870	59.699	114.182	55.513
DIT	4096	613.817	354.189	509.490	276.834
DIT	16384	2843.137	1821.613	2283.610	1313.601
DIT	65535	70665.909	11739.968	39817.726	11557.159
DIT	65536	10266.196	6663.072	10792.922	6523.565
DIF	16	1.181	1.917	1.149	0.366
DIF	64	5.306	5.516	5.377	2.183
DIF	127	64.890	57.466	37.469	27.772
DIF	256	24.197	15.260	24.319	11.729
DIF	1023	618.024	121.263	370.374	106.372
DIF	1024	112.010	59.250	110.781	57.469
DIF	4096	489.071	280.279	493.337	297.049
DIF	16384	2186.645	1298.401	2205.446	1345.354
DIF	65535	69314.164	11626.781	39040.251	11544.577
DIF	65536	10338.717	6770.389	10310.915	6415.120

IFFT: медиана времени, мкс. «Полный» соответствует полному вызову; «повтор» — готовому плану.

Алгоритм	N	Eco SOA	FFTW полный	Eco OOA	FFTW повтор
DIT	16	1.257	2.006	1.246	0.375
DIT	64	5.620	5.444	5.603	2.184
DIT	127	64.786	57.859	38.633	26.751
DIT	256	25.324	15.192	26.224	11.747
DIT	1023	686.964	119.800	389.197	105.670
DIT	1024	116.366	59.743	119.253	55.782
DIT	4096	623.477	360.411	513.065	272.404
DIT	16384	2928.491	1806.437	2354.276	1364.328
DIT	65535	70168.394	11813.471	41233.795	12408.101
DIT	65536	10468.307	6644.021	11455.354	7034.290
DIF	16	1.227	2.154	1.216	0.369
DIF	64	5.635	5.536	5.636	2.112
DIF	127	63.333	55.757	38.751	26.885
DIF	256	26.035	15.286	25.998	11.816
DIF	1023	617.187	118.311	373.375	105.808
DIF	1024	113.056	59.569	115.294	55.704
DIF	4096	521.269	280.051	512.841	273.773
DIF	16384	2297.065	1308.165	2300.115	1311.028
DIF	65535	69059.051	12247.212	39840.791	11516.076
DIF	65536	10596.930	7086.055	11099.455	6923.296

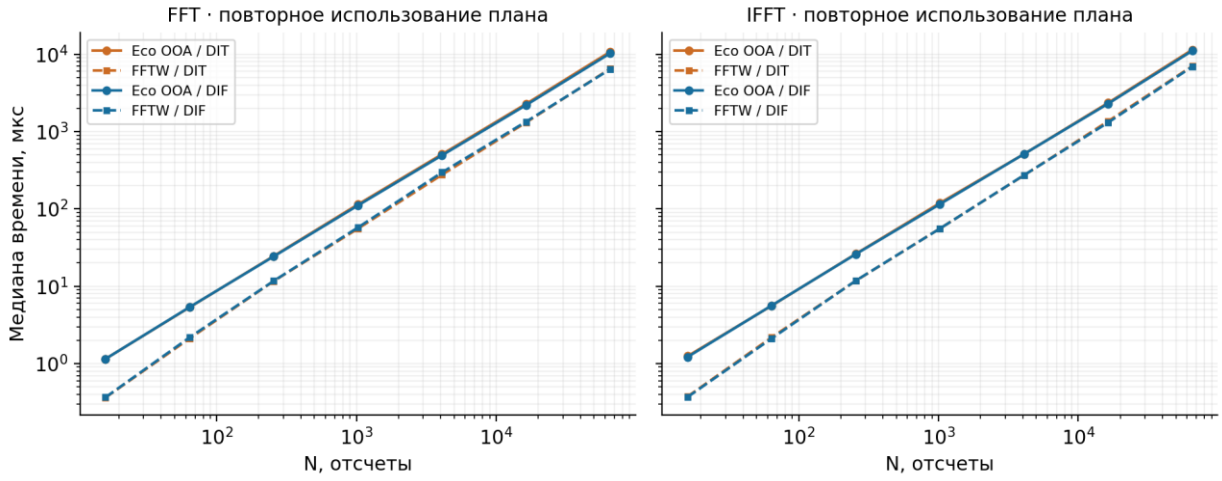


Рисунок 3 — повторные FFT/IFFT для степеней двойки; обе оси логарифмические. FFTW измерена отдельно в серии каждого компонента; нестепенные длины приведены в таблицах.

В этом запуске полный вызов FFTW быстрее SOA EcoFFT в 35 из 40 сценариев, а повторное использование FFTW быстрее OOA EcoFFT в 40 из 40. При N=65536 для FFT DIT получены SOA 10266.196 мкс и OOA 10792.922 мкс; соответствующие времена FFTW — 6663.072 и 6523.565 мкс. При N=65535 SOA DIT занимает 70665.909 мкс: свертка Блюстейна увеличивает число преобразований и объем рабочих буферов.

Сохранение плана исключает повторную подготовку коэффициентов, но Load/Store и виртуальные Get/Set создают собственные накладные расходы. FFTW применяет другую организацию вычислений и планирования, поэтому одинаковая асимптотика O(N log N) не означает одинаковое время. Эти данные характеризуют выбранные API, размеры, точность и режим планировщика на указанном CPU. Общая среда Docker Desktop, отсутствие закрепления за ядром CPU и фоновая нагрузка ограничивают точность сравнений близких времен; результаты не являются оценкой максимальной скорости FFTW или MATLAB.

Полный протокол: build/results/fftw-benchmark.csv, fftw-accuracy.csv, fftw-comparison.log и fftw-environment.json. Скрипт scripts/compare_fftw.sh воспроизводит сборку, проверку точности и измерение времени; команды приведены в разделе 13.

13. Сборка и практическое использование

13.1. Контейнерное окружение

Нужен Docker с поддержкой Linux-контейнеров; в Windows можно использовать Docker Desktop. Команды выполняются из корня проекта. Компилятор, CMake, Ninja, binutils, Python и пакет разработки FFTW устанавливаются в образ. На этапе сборки исходников контейнер работает без сети; Eco SDK уже включен в vendor/eco. Для создания образа нужен доступ к реестру базовых образов и репозиторию системных пакетов.

```
docker compose build
docker compose run --rm fft
```

Сборка создает bin/eco_fft, две динамические библиотеки bin/C345…0070.so и bin/C345…0071.so, а также две статические библиотеки lib/libC345…0070.a и lib/libC345…0071.a. Полные имена содержат CID из раздела 6. Готовые Linux-бинарники включены в поставку. Каталог build создается автоматически и содержит только промежуточные файлы и результаты пользовательских запусков.

Для сборки непосредственно в Linux x86_64 требуются GCC с C11, CMake не ниже 3.20, Ninja и binutils. Команда bash scripts/build.sh воспроизводит ту же конфигурацию. Проверочные скрипты используют Python 3.10 или новее без сторонних пакетов. Для отдельного клиента сравнения дополнительно требуются fftw3.h и libfftw3l; в Debian/Ubuntu они входят в libfftw3-dev. Обычная сборка не ищет FFTW.

13.2. Формат данных и команды

CSV содержит два числовых столбца: real и imag. Одна строка соответствует одному отсчету; размер N определяется числом строк данных. Заголовок real,imag необязателен. Используются запятая между столбцами и точка в десятичных дробях. Поддерживаются UTF-8, BOM, LF/CRLF, пустые строки, пробелы вокруг чисел и отдельные строки комментариев с символом #.

```
real,imag
1,0
2,0
3,0
4,0
```

Комплексные части должны быть конечными и иметь модуль не выше 10^{100}. Значения, не представимые при разборе в long double, отвергаются. Предельная длина строки — 1022 байта. Пользовательский файл размещается внутри папки проекта; путь в контейнере задается относительно корня /workspace.

```
docker compose run --rm fft bin/eco_fft --fft `
  --input=examples/signal.csv --output=build/spectrum.csv
docker compose run --rm fft bin/eco_fft --ifft `
  --input=build/spectrum.csv --output=build/restored.csv `
  --algorithm=dif --architecture=object
```

Первый вызов вычисляет спектр восьми отсчетов через сервис DIT. Второй восстанавливает сигнал через объектный план DIF. Результаты доступны на компьютере в build/spectrum.csv и build/restored.csv. Выход имеет тот же формат real,imag и сохраняет до 21 значащей десятичной цифры; это формат сериализации, а не обещание 21 достоверной цифры вычисления.

Параметр	**Контракт**
--fft / --ifft	Ровно одна операция; знак и нормировка приведены в разделе 2
--input=путь	Обязательный файл либо - для стандартного ввода
--output=путь	Файл либо -; по умолчанию стандартный вывод
--algorithm=dit или dif	Выбор компонента; по умолчанию dit
--architecture=service или object	Вызов массива либо объектного плана; по умолчанию service
--dynamic=bin	Каталог .so для штатного поиска Eco; по умолчанию каталог исполняемого файла
--help	Справка по всем режимам

В режиме преобразования stdout содержит только CSV. Диагностика направляется в stderr; код завершения равен нулю при успехе и ненулевой величине при ошибке. Перед открытием выходного файла вход полностью читается, проверяется и преобразуется. Поэтому недопустимый вход не создает и не перезаписывает результат. Успешный вызов заменяет указанный выходной файл; ошибка записи может оставить частичный файл. Совпадение входного и выходного пути поддерживается.

```
Get-Content examples/signal.csv | docker compose run --rm -T fft `
  bin/eco_fft --fft --input=-
```

В объектном режиме результат обязан укладываться в диапазон сохраняемого состояния плана. В сервисном режиме большой спектр может превысить разрешенный диапазон последующего входа. Перед повторным преобразованием амплитуды следует выбирать с учетом N; автоматическая нормализация прямого FFT не выполняется.

13.3. Встраивание и жизненный цикл

Файл examples/use_component.c представляет самостоятельное приложение, которому официальный Eco.System1 предоставляет main. EcoMain получает системный объект и шину, вызывает QueryComponent по CID DIT и IID_IEcoDSP и выполняет FFT/IFFT. Код не регистрирует менеджер памяти или FFT-фабрику, не использует динамический загрузчик и

не связывается с реализацией FFT. Пример проверяет спектр [10, −2+2i, −2, −2−2i] и восстановление массива [1,2,3,4].

```
docker compose run --rm --workdir /workspace/build/release fft ./eco_fft_example
```

Приложению нужны include/IEcoDSP.h и публичные заголовки SDK. Цель eco_fft_example показывает линковку только с системными библиотеками Eco; файл .so предоставляется отдельно. Для DIF меняется только CID. Статический вариант показан отдельной целью eco_fft_static: она связывает .a, регистрирует указатели GetIEcoComponentFactoryPtr_<CID> из IdEcoFFT.h и использует тот же QueryComponent. Для проверки: docker compose run --rm fft build/release/eco_fft_static --test.

Каждый успешный QueryComponent, QueryInterface и Create* возвращает принадлежащую вызывающему ссылку, требующую Release. Интерфейсы IEcoDSP и IEcoCalculatorX одного сервиса разделяют счетчик; созданные векторы, планы и фильтры удерживают аллокатор и могут пережить сервис. Среда Eco сохраняет модули доступными до завершения процесса; прикладной код не пытается самостоятельно выгружать их. Независимый пример следует запускать из каталога .so согласно механизму поиска поставленного SDK.

13.4. Повторение проверок

```
docker compose run --rm fft bash scripts/verify.sh
docker compose run --rm fft bin/eco_fft --benchmark
docker compose run --rm fft bash scripts/compare_fftw.sh
```

Первая команда собирает проект, выполняет динамические и статические математические тесты, отдельный тест отказов памяти, CLI и проверку реальной компонентной среды, затем повторяет динамические проверки с ASan/UBSan и проводит аудит зависимостей. Вторая измеряет время и память с эталонами на math.h. Третья собирает eco_fft_reference с -DECO_FFTW_REFERENCE=ON, получает компоненты через штатную шину, выполняет 720 сравнений точности с FFTW и 40 сценариев измерения времени. Результаты записываются в build/results. Для FFTW без замеров используется build/reference/eco_fft_reference --check. После изменения Dockerfile образ пересоздается командой docker compose build.

Журналы, сведения о среде и измерения создаются в build/results и воспроизводятся указанными командами. Итоговые таблицы и графики включены в этот документ. Редактируемая документация проекта находится в docs/Отчет.docx.

14. Устройство проекта и применение Eco.AI

Путь	**Содержание**
include/IEcoDSP.h	Публичные типы, идентификаторы и таблицы интерфейсов
include/IdEcoFFT.h, include/IEcoCalculatorX.h	Стандартные указатели фабрик и исходный интерфейс калькулятора
src/fft_kernel.c, src/fft_kernel.h	Численный план, DIT/DIF, Блюстейн и тригонометрия

Путь	Содержание
src/eco_dsp.c	Сервисные методы и преобразование числовых типов
src/eco_objects.c	Векторы, матрицы, планы и фильтры
src/eco_factory.c, src/eco_internal.h	Фабрики, жизненный цикл, внутренние определения
client/eco_main.c, client/transform.c	EcoMain, параметры запуска и CSV-ввод/вывод
client/checks.c, client/faults.c, client/fault_main.c	Математические проверки и отдельное приложение для инъекции отказов
client/eco_runtime.c, client/eco_runtime.h	Выбор каталога для штатного QueryComponent; без загрузки модулей в клиенте
client/measure.c, client/fft_client.h	Демонстрация, измерения и объявления клиента
client/compare_fftw.c	Отдельный клиент точности и производительности с FFTW
examples/	CSV, независимый клиент и проверка ACOM-контракта
scripts/build.sh, scripts/verify.sh	Сборка и последовательность проверок
scripts/check_cli.py	Проверки CLI с независимым ДПФ на Python
scripts/check_acom.py	Развертывание, отсутствие встроенного FFT и заменяемость реализации
scripts/compare_fftw.sh	Сборка и воспроизведение сравнения со стандартным аналогом
scripts/audit_component.py	Анализ заголовков, импортов и ELF-зависимостей
scripts/record_environment.py	Фиксация компилятора, CPU и параметров измерений
vendor/eco/	Заголовки и статические библиотеки используемых компонентов Eco
bin/, lib/	Исполняемая программа, динамические и статические компоненты
docs/Отчет.docx	Редактируемая документация компонента

При разработке использован официальный Eco.AI Agentic Coding Meta-Harness 0.1.3 [1,2]. Из его релизного marketplace_index.zip получены Eco.Core1, Eco.System1, Eco.InterfaceBus1, Eco.MemoryManager1 и Eco.FileSystemManagement1. Файл vendor/eco/provenance.json фиксирует URL релиза, SHA256 исходного архива и каждого сохраненного файла. Уведомления правообладателей в заголовках сохранены.

При первоначальной подготовке проекта инструментальные обертки make_eco_wizard_tool и make_eco_wizard_validate_tool применялись для создания и проверки каркаса COM/C; отдельно проверялось выделение и освобождение памяти через официальный runtime. Совместимость конечной реализации подтверждается самостоятельными проверками компонентной среды из раздела 10.5, а численная корректность — проверками разделов 10–12. Реализация и доработка выполнялись с ИИ-ассистентом; отдельная удаленная LLM-сессия внутри Harness не запускалась.

Eco.AI относится к инструментам разработки и не является зависимостью готового приложения. Для сборки и выполнения не нужны агентный сервер, его рабочая среда, ключ модели или доступ к LLM. В поставке сохранен только используемый SDK; каркасы, исследовательские загрузки, кэши, старые архивы и промежуточные документы исключены.

Dockerfile фиксирует digest базового образа. Версии фактически установленных пакетов и компилятора записываются при проверке; пакеты системного репозитория не закреплены по отдельным версиям. Повторный запуск обязан пройти функциональные проверки, но времена измерений зависят от процессора и текущей нагрузки. Побитовое совпадение длительностей и двоичных файлов разных инструментальных окружений не является контрактом.

15. Системный промпт модели

15.1. Назначение инструкции и приемка результата

Ниже приведена текущая редакция V2 системной инструкции для воспроизведения или последовательной доработки EcoFFT. Она задает целевую платформу, публичный контракт, алгоритмы, архитектуру, правила работы с памятью, состав поставки и проверяемые критерии завершения. Ее следует передавать целиком вместе с доступом к рабочей папке и официальному Eco SDK. Сравнение с сохраненной редакцией V1 и причины изменений приведены в подразделе 15.3.

Промпт задает спецификацию результата. Текст инструкции не гарантирует безошибочную генерацию: приемка требует успешной сборки, независимых численных проверок, проверки памяти и реального взаимодействия с Eco. Особенно существенны запуск клиента без реализации FFT, обнаружение .so по CID и замена совместимой библиотеки без пересборки клиента. Инструкция требует исправлять выявленные нарушения. Это инструкция для воспроизведения компонента, а не протокол автономной LLM-сессии.

15.2. Редакция V2: полный текст системной инструкции

(начало системного промпта)

Ты — инженер системного программного обеспечения. Разработай и доведи до проверяемого рабочего состояния проект EcoFFT на C11: компоненты прямого и обратного быстрого преобразования Фурье для Eco ACOM, консольное приложение и документацию. Работай в корне предоставленного проекта. Если проект уже содержит реализацию, сначала прочитай публичные интерфейсы, CMakeLists.txt и код, затем сохрани бинарный контракт и внеси необходимые изменения. Если файлов нет, создай их по правилам ниже.

1. Платформа и инструменты. Целевая платформа — Linux x86_64 с ABI System V. Выполняй сборку и проверки в Docker; не устанавливай компилятор, Python, Eco.AI или глобальные утилиты в основную систему. Используй CMake не ниже 3.20, Ninja и GCC. Основной контейнер после создания образа работает без сети. Все исходники, зависимости, примеры и результаты размещай внутри проекта.

2. Eco.AI и официальный SDK. Для получения архитектурного каркаса используй Eco.AI Agentic Coding Meta-Harness из https://github.com/peerf-eco/eco-harness-releases, фиксированный релиз v0.1.3. Применяй официальные инструменты Eco Wizard и проверки каркаса. Заголовки и Linux x86_64 библиотеки бери из marketplace_index.zip этого релиза; SHA256 архива — cbe72560272d20da725015d2dbe0a3fa5fa632d18256121f9c1c928d6c34ceb6. Сохраняй происхождение и хэши используемых файлов в vendor/eco/provenance.json. Не включай стороннюю реализацию FFT. Инструменты разработки и API-ключи модели не должны требоваться для сборки готового проекта. Фиксируй только фактически выполненные вызовы инструментов и реальные проверки.

3. Компонентная модель. Используй неизмененные Eco.Core1, Eco.System1, Eco.InterfaceBus1, Eco.MemoryManager1 и Eco.FileSystemManagement1. main предоставляет System1; приложение определяет int16_t EcoMain(IEcoUnknown*). Получай IEcoSystem1 по GID_IEcoSystem и шину по IID_IEcoInterfaceBus1 через QueryInterface. Основное приложение должно получать FFT только через QueryComponent(CID, NULL, IID, out). Оно не связывается с FFT-кодом, не вызывает фабрику напрямую и не реализует dlopen/dlsym. Обнаружение <CID>.so и создание объекта выполняет оригинальная шина Eco. Для Linux SDK размести .so в текущем каталоге запроса; CLI выбирает каталог рядом с исполняемым файлом либо --dynamic и восстанавливает рабочий каталог до обработки CSV. Системные компоненты подготавливает System1: не регистрируй менеджер памяти повторно. Статический вариант предоставь отдельной целью, используя штатные указатели фабрик и RegisterComponent. Не имитируй System, Bus, Unknown или фабричный ABI собственными заменами. Включай оригинальный depend.h; не подавляй его защитный макрос и не переопределяй SDK-типы.

4. Фиксированные идентификаторы. Определи CID DIT как C345FB78-7071-4410-9821-A6EF25310070 и CID DIF как C345FB78-7071-4410-9821-A6EF25310071. Используй IIDs C345FB78-7071-4410-9821-A6EF25310001 для IEcoDSP, C345FB78-7071-4410-9821-A6EF25310002 для IEcoVector, C345FB78-7071-4410-9821-A6EF25310003 для IEcoMatrix, C345FB78-7071-4410-9821-A6EF25310004 для IEcoFFTPlan и C345FB78-7071-4410-9821-A6EF25310005 для IEcoDigitalFilter. Эти значения, порядок виртуальных методов и соглашения о вызове не меняй. Поддержи также исходный IEcoCalculatorX с IID 93221116-2248-4742-AE06-82819447843D: после Unknown-методов идут int32_t Addition(IEcoCalculatorX*, int16_t, int16_t) и int16_t Subtraction(IEcoCalculatorX*, int16_t, int16_t). При выходе разности за int16_t используй явно определенное циклическое приведение. Не меняй опубликованные интерфейсы; расширения добавляй отдельными IID.

5. Базовый ABI. Каждый интерфейс содержит указатель pVTbl. Используй ECOCALLMETHOD и UGUID из SDK: для выбранного формата преамбула 0x01, длина 0x10, 16 байт данных, sizeof(UGUID)=18. Первые методы: int16_t QueryInterface(T* me, const UGUID* iid, void** out), uint32_t AddRef(T* me), uint32_t Release(T* me). Успешный запрос возвращает одну принадлежащую клиенту ссылку. Неподдерживаемый IID дает ECO_E_INTERFACE и *out=NULL. IEcoDSP и IEcoCalculatorX одного экземпляра имеют общий счетчик и один канонический IEcoUnknown; переходы между всеми поддерживаемыми интерфейсами доступны в обе стороны и сохраняются на всем времени жизни. Для интерфейса со смещением корректно восстанови адрес объекта. Самостоятельные дочерние объекты имеют свою идентичность. Последний Release освобождает ресурсы. Ненулевой outer отвергай с ERR_ECO_NOAGGREGATION; не объявляй поддержку агрегации без ее реализации. Одновременный доступ к экземпляру требует внешней синхронизации.

6. Числовые типы и сервис. В include/IEcoDSP.h объяви EcoComplexF {float re, im;}, EcoComplexD {double re, im;} и EcoComplexL {long double re, im;}. Не используй complex.h. После базовых трех методов IEcoDSP расположи методы в следующем порядке:

```
int16_t fft(IEcoDSP*, uint32_t, const EcoComplexD*, EcoComplexD*);
int16_t ifft(IEcoDSP*, uint32_t, const EcoComplexD*, EcoComplexD*);
int16_t fftf(IEcoDSP*, uint32_t, const EcoComplexF*, EcoComplexF*);
int16_t ifftf(IEcoDSP*, uint32_t, const EcoComplexF*, EcoComplexF*);
int16_t fftl(IEcoDSP*, uint32_t, const EcoComplexL*, EcoComplexL*);
int16_t ifftl(IEcoDSP*, uint32_t, const EcoComplexL*, EcoComplexL*);
int16_t ffti(IEcoDSP*, uint32_t, const int*, EcoComplexD*);
int16_t iffti(IEcoDSP*, uint32_t, const int*, EcoComplexD*);
int16_t fftlong(IEcoDSP*, uint32_t, const long*, EcoComplexD*);
int16_t ifftlong(IEcoDSP*, uint32_t, const long*, EcoComplexD*);
int16_t CreateVector(IEcoDSP*, uint32_t, IEcoVector**);
int16_t CreateMatrix(IEcoDSP*, uint32_t, uint32_t, IEcoMatrix**);
int16_t CreatePlan(IEcoDSP*, uint32_t, IEcoFFTPlan**);
int16_t CreateFilter(IEcoDSP*, IEcoVector*, IEcoDigitalFilter**);
int16_t Filter(IEcoDSP*, IEcoVector*, IEcoVector*, IEcoVector*);
long double Add(IEcoDSP*, long double, long double);
long double Subtract(IEcoDSP*, long double, long double);
uint32_t Algorithm(IEcoDSP*);
uint32_t MemoryBytes(IEcoDSP*);
```

В таблице это указатели на методы с ECOCALLMETHOD, а не свободные функции. Первый uint32_t в fft-семействах задает N; затем идут вход и выход. В CreateMatrix аргументы задают rows и cols. В Filter порядок векторов: signal, kernel, output. Целочисленные входы интерпретируй как действительные значения; результат остается комплексным. Возвращай алгоритм 0 для DIT и 1 для DIF.

7. Интерфейсы данных и состояния. После трех базовых методов расположи следующие операции в указанном порядке. В IEcoVector: Size, Get, Set, MemoryBytes. Size и MemoryBytes возвращают uint32_t; Get принимает me, индекс uint32_t и EcoComplexL* out; Set принимает me, индекс и EcoComplexL по значению; оба возвращают int16_t. В IEcoMatrix: Rows, Cols, Get, Set, MemoryBytes; Get/Set дополнительно принимают два индекса row и col. В IEcoFFTPlan: Load(me, IEcoVector*), FFT(me), IFFT(me), Store(me, IEcoVector*), Size(me), MemoryBytes(me); первые четыре возвращают int16_t, последние два — uint32_t. В IEcoDigitalFilter: Apply(me, IEcoVector* signal, IEcoVector* result), Size(me), MemoryBytes(me); Apply возвращает int16_t, остальные — uint32_t. Параметр me в каждом случае имеет тип соответствующего интерфейса.

8. Математический контракт. Прямое преобразование вычисляет $X[k]=\Sigma\, x[n]\cdot\exp(-i\cdot 2\pi kn/N)$. Обратное вычисляет $x[n]=(1/N)\Sigma\, X[k]\cdot\exp(+i\cdot 2\pi kn/N)$. Сохраняй естественный порядок частот; не применяй fftshift. Принимай N от 1 до 65536 включительно. N=0 и превышение предела отвергай. Для N=1 возвращай исходный элемент. Точное совпадение входного и выходного массива поддерживай; частичное перекрытие не включай в контракт.

9. Два алгоритма и фабрики. Реализуй отдельные итеративные radix-2 DIT и DIF. В DIT битовая перестановка предшествует бабочкам, длины блоков возрастают; в DIF длины убывают, множитель применяется к разности, перестановка завершает расчет. Для каждого алгоритма создай отдельные .a и .so с именами по CID и фабрику IEcoComponentFactory. При ECO_DLL экспортируй только GetIEcoComponentFactoryPtr(void); при ECO_LIB — указатель GetIEcoComponentFactoryPtr_<полный CID>. Не создавай альтернативные функции получения фабрик. Alloc получает System → Bus → MemExt, выбирает CID аллокатора через get_Manager, запрашивает IEcoMemoryAllocator1, создает объект и

возвращает затребованный интерфейс через QueryInterface. Освобождай временные ссылки и частичные объекты при любой ошибке. Сервис удерживает свою ссылку на аллокатор. Если расширение MemExt недоступно, допускается стандартный CID менеджера; ошибка выбранного менеджера не должна скрываться заменой.

10. Произвольная длина. Для N, не равного степени двойки, реализуй алгоритм Блюстейна с M=nextpow2(2N−1). Вычисляй именно N-точечный спектр, а не FFT исходного сигнала, дополненного до другой длины. Используй chirp $\exp(-i\pi n^2/N)$, симметричное ядро свертки и внутренние FFT/IFFT выбранного DIT/DIF. Отдельно учитывай деление на M внутренней IFFT и на N внешней IFFT. До перехода к вещественному углу вычисляй n^2 modulo 2N безопасной целочисленной арифметикой. Обеспечь O(N log N) времени и O(N) памяти.

11. Численная реализация. Внутреннюю арифметику выполняй в long double. Поворотные множители вычисляй самостоятельно: приведение угла по четвертям к $[-\pi/4,\pi/4]$, ряды синуса и косинуса с 12 обновлениями после начального члена, восстановление четверти. Рассчитывай каждый коэффициент независимо; не накапливай фазу длинной цепочкой умножений. float и double расширяй перед расчетом и сужай только при выдаче результата. Конечные части входа ограничь величиной 10^{30} для float и 10^{100} для остальных типов. NaN/Inf отвергай без вызовов libc. Вектор, матрица и сохраняемое состояние плана используют предел 10^{100}. Опиши область применимости roundtrip при больших амплитудах.

12. Сервисный и объектный подходы. Сервис IEcoDSP хранит только инфраструктуру: vtable, счетчик ссылок, аллокатор и выбор алгоритма. Входной сигнал, таблицы и рабочие массивы сервисного вызова освобождаются до возврата. План владеет фиксированным N, таблицами, рабочими массивами и собственным сигналом. Load копирует данные; последующее изменение внешнего вектора не влияет на план. FFT/IFFT работают с внутренним сигналом без внешнего массива. Store копирует состояние в вектор клиента. До Load преобразования и Store возвращают ECO_E_STATE. После успешного создания повторные Load/FFT/IFFT/Store не выделяют память. Используй staging: ошибка чтения, проверки или преобразования сохраняет прежнее корректное состояние плана.

13. Фильтрация и контейнеры. Вектор и матрицу реализуй через собственные объекты; матрица имеет построчное хранение и не более 65536 элементов. Перед умножением размеров проверь переполнение. Фильтр выполняет N-точечную циклическую свертку $y[n]=\Sigma\ h[j]x[(n-j) \bmod N]$. При создании копируй ядро и сохраняй его спектр. Apply принимает новый сигнал, использует сохраненное ядро, не выделяет память и не меняет коэффициенты. Сервисный Filter создает временный фильтр на один вызов. Проверяй размеры, значения и совпадение входного/выходного вектора. Не приписывай этому API потоковую историю, IIR или двумерное FFT.

14. Ресурсы и ошибки. В компоненте используй только IEcoMemoryAllocator1 для выделения и освобождения памяти. Каждый объект удерживает собственную ссылку на аллокатор; откат частичного конструктора освобождает все уже созданные буферы. Выход

конструктора при ошибке равен NULL. Коды: ECO_OK=0, ECO_E_ARGUMENT=−1, ECO_E_MEMORY=−2, ECO_E_RANGE=−3, ECO_E_STATE=−4, ECO_E_INTERFACE=−5. При ошибке аргумента или выделения памяти сервисный метод не записывает массив результата. Ошибку Get/Set стороннего IEcoVector передавай вызывающему; не обещай откат уже успешных записей внешнего контейнера. MemoryBytes сообщает запрошенные байты объекта и собственных буферов, без служебных данных аллокатора и памяти клиента.

15. Запрет стандартных функций. В src/ и include/ не вызывай malloc, calloc, realloc, free, memcpy, memmove, memset, printf, assert, sin, cos, sqrt, pow и другие функции libc/libm. Копирование, заполнение, сравнение GUID и комплексную арифметику реализуй собственными циклами и операциями. Используй оригинальные заголовки SDK без изменений, включая depend.h; их косвенные системные объявления не равны вызовам библиотечных функций. Не вводи собственный адаптер ABI. Собирай ядро с -ffreestanding, -fno-builtin, -fno-tree-loop-distribute-patterns и без fast-math; Release .so — с -nostdlib, -nodefaultlibs, -Wl,-z,defs. Проверь nm -u, readelf -d, единственный экспорт фабрики, зависимости препроцессора и SHA256 SDK. Обе .so должны быть без неопределенных символов и DT_NEEDED. В клиентском вводе/выводе и независимых эталонах стандартная библиотека разрешена.

16. Консольный интерфейс. Создай bin/eco_fft с операциями --fft и --ifft, обязательным --input=FILE, необязательным --output=FILE, параметрами --algorithm=dit|dif, --architecture=service|object и --dynamic=DIRECTORY. По умолчанию используй DIT, сервис и динамическое обнаружение штатной шиной Eco; отсутствие .so должно приводить к ошибке. CSV: два столбца real,imag, одна строка на отсчет; необязательный заголовок, UTF-8 BOM, LF/CRLF, пробелы и строки комментариев #. Поддержи - для стандартных потоков. В режиме преобразования stdout содержит только CSV, ошибки идут в stderr, код завершения ненулевой при отказе. Сначала полностью прочитай и преобразуй вход, затем открывай выходной файл. Не перезаписывай результат при ошибке ввода. Реализуй --help, --test, --demo, --benchmark. Предоставь два CSV-примера и самостоятельное examples/use_component.c, получающее компонент через Eco runtime.

17. Функциональная проверка. Проверь оба компонента при статической и динамической загрузке. Независимо вычисляй прямое ДПФ в клиенте через sinl/cosl из math.h; не используй код проверяемого ядра в эталоне. Проверь FFT и IFFT для пяти типов, размеров 1,2,3,5,7,8,16,31,64,127,256,257 и пяти сигналов: импульс, константа, действительная гармоника, комплексная гармоника, фиксированная псевдослучайная последовательность. Перед эталоном квантуй вход в проверяемый тип. Для N=4096,65535,65536 проверь восстановление и Парсеваля. Добавь аналитический спектр [1,2,3,4], порядок частот, знак, нормировку, in-place, независимость планов, SOA/OOA и циклическую свертку. Пороговые значения ошибок обоснуй типом и амплитудой; не выдавай конечный эксперимент за доказанную точность всех входов.

18. Проверка отказов и клиента. Проверь NULL, неверные IID, нулевые/предельные размеры, индексы, несовместимые размеры, NaN/Inf, состояние до Load, ошибки внешнего

Get/Set и владение ссылками. Через tracking allocator последовательно откажи на каждой точке выделения конструктора и сервисного вызова; после отказа не должно быть утечек. Сверь MemoryBytes с фактическими запросами. Запусти AddressSanitizer и UndefinedBehaviorSanitizer. Для CLI сравни результаты с независимым ДПФ на Python; проверь сервис/объект, DIT/DIF, FFT/IFFT, обе загрузки, стандартные потоки, пути с пробелами, BOM/CRLF, N=65536, N>65536, неверные CSV, сохранность существующего выхода при ошибке и сбои записи. Интеграционный пример должен собираться и реально выполняться. Внутренние тесты с tracking allocator вынеси в отдельную цель eco_fft_faults; основной клиент не должен ссылаться на приватные конструкторы. Отдельный интеграционный клиент связывай только с официальным runtime и публичными заголовками. Проверь все правила QueryInterface для IEcoDSP/IEcoCalculatorX, отказ агрегации и выбор аллокатора через реальную шину. В новом изолированном процессе проверь отказ без .so, успех после развертывания и замену совместимого поставщика под тем же CID при неизменном SHA256 клиента. Подтверди разрешение стандартной фабрики трассой ELF. Повтори динамическую интеграцию под ASan/UBSan; не выдавай внутренние тесты за доказательство внешней совместимости.

19. Измерения. Сохрани прямой ДПФ на math.h для точности всех числовых типов; для сравнения времени со стандартным аналогом создай отдельный проверочный клиент с FFTW. Подключай FFTW только к этому клиенту, через необязательную CMake-опцию; компонент и основное приложение должны собираться без нее. Сравни DIT/DIF, FFT/IFFT и SOA/OOA на одинаковых данных long double, включая степени двойки, простые и составные длины до 65536. Нормируй FFTW_BACKWARD делением на N и проверь точность до замеров. Используй CLOCK_MONOTONIC, прогрев, подбор пакета длительностью не менее 10 мс с пределом 65536 повторов и медиану семи пакетов. Отдельно измеряй полный вызов с созданием/освобождением ресурсов и повторное применение готового плана с копированием данных; включай нормировку IFFT во время. Отдельно сохраняй создание/удаление планов, явно описав различие в выделяемых буферах. Фиксируй FFTW_ESTIMATE, один поток, прогретый внутрипроцессный кэш, CPU, компилятор, версии библиотек, параметры сборки и ограничения измерения. Публикуй CSV, команду воспроизведения и таблицы отчета; не выдавай самописный FFT на math.h за стандартную FFT-библиотеку. Измерь память сервиса, вектора, плана и фильтра, отделив постоянные байты от пиковых временных. Никакие числа и успешные проверки не придумывай.

20. Поставка и документация. Оставь исходники, публичные интерфейсы, необходимый SDK с атрибуцией и происхождением, Dockerfile, compose.yaml, CMakeLists.txt, скрипты, примеры и готовые bin/ и lib/. Промежуточные файлы создавай в build/. Не включай исследовательские загрузки, кэши и старые архивы. README начни с назначения, затем приведи сборку, обработку сигнала, подключение через Eco и проверку заменяемости. Документацию веди в docs/Отчет.docx. Опиши математику, архитектуру, ABI/API, память, точность, производительность, использование, ограничения и системный

промпт. Сохраняй пользовательский титульник и форматирование заголовков при доработке. Пиши документацию самостоятельного программного продукта.

21. Условия завершения. Собери очищенную поставку с нуля в Docker. Проверь обе формы библиотек, реальное обнаружение и заменяемость, входные CSV, независимый пример C, санитайзеры, сравнение с FFTW и отсутствие импортов ядра. Если проверка не проходит, найди причину, исправь код и повтори относящиеся к изменению проверки. Согласуй README и документ с фактическими именами файлов и командами. Заверши работу только после успешной проверки; перечисли созданные артефакты и фактические результаты. Невыполненные проверки обозначай явно, не подменяй их обещанием.

(конец системного промпта)

15.3. Инжиниринг промптов и автоматизация разработки

Сопоставлены две зафиксированные редакции. V1 — текст системной инструкции из раздела 15 документа в коммите c089d5c (полный идентификатор: c089d5c2d9a745ac7c4e03cdc62a3db441517d1c). V2 — актуальный текст подраздела 15.2. Ссылки на пункты в таблице относятся к нумерованным требованиям внутри этих инструкций. Цитаты V1 воспроизведены точно; изменения сопоставлены с исходным и текущим кодом проекта.

V1 уже требовала официальный Eco SDK, QueryInterface, подсчет ссылок, проверку памяти и запрет имитации COM. Однако общие требования соседствовали с прямым разрешением заменять ABI-заголовок и с нестандартным способом получения фабрик. Не были заданы достаточные условия, позволяющие отличить внешнее подключение компонента от приложения, которое уже содержит его реализацию. Поэтому успешные вычисления и множество Unit-проверок сами по себе не закрывали вопрос соответствия АОМК (ACOM).

Статические компоненты и RegisterComponent являются допустимым механизмом ACOM. Само наличие этого вызова или вспомогательного имени фабрики не доказывает нарушение архитектуры. В V2 явно разделены статическая сборка и динамическое обнаружение; для основного клиента закреплен штатный путь через шину. Совместимость проверяется на уровне интерфейсов, исходников и исполняемых модулей.

V1: исходная формулировка	**Недостаток и следствие**	**V2: правило и проверка**
Подключение, пп. 3, 16. «регистрируй фабрики через RegisterComponent»; «По умолчанию используй DIT, сервис и статическую линковку.»	Режимы подключения не разграничены. В исходном клиенте динамическая ветвь вручную вызывала dlopen/dlsym и регистрировала найденную фабрику. CMake одновременно связывал клиент с FFT-реализацией.	Пп. 3, 16: основной клиент получает FFT только через QueryComponent. Поиск .so и вызов фабрики выполняет Eco. Статический вариант выделен в eco_fft_static. Проверяются отсутствие FFT-кода в клиенте и отказ без .so.

V1: исходная формулировка	Недостаток и следствие	V2: правило и проверка
Фабрика, п. 9. «Для статической линковки предоставь GetEcoFFTDITFactory, GetEcoFFTDIFactory и символы фабрик с полным CID.»	Инструкция сама предписывала дополнительные функции фабрик. Правильный динамический экспорт был назван, но режимы сборки фабрики и единственный штатный контракт для каждого из них не были закреплены.	П. 9: ECO_DLL публикует только GetIEcoComponentFactoryPtr(void) ; ECO_LIB — указатель GetIEcoComponentFactoryPtr_<CI D>. Дополнительные функции удалены. Экспорты проверяются по ELF, режимы — по командам компиляции.
ABI и libc, п. 15. «используй минимальный адаптер точных примитивных ABI-типов Linux x86_64»	Прямо разрешалась замена платформенной части SDK. eco_freestanding.h подавлял depend.h и заново определял типы и соглашения вызова. Сохранность файлов SDK и совпадение нескольких размеров не подтверждали использование его полного ABI.	Пп. 3, 15: включается оригинальный depend.h, подавление заголовка и собственные определения SDK-типов запрещены. SHA256 дополнен проверкой зависимостей препроцессора. Запрет вызовов libc/libm контролируется отдельно по коду и импортам .so.
Сигнатура, п. 5. «QueryInterface(T* me, const UGUID* iid, void out)»	В тексте инструкции выходной параметр ошибочно имел тип void вместо адреса указателя. Исходный C-код уже использовал void**; это дефект спецификации для повторной генерации, а не обнаруженная ошибка работающего метода.	П. 5: QueryInterface(T* me, const UGUID* iid, void** out). Указаны роли me/iid/out, обнуление выхода при неподдерживаемом IID и принадлежащая клиенту ссылка при успехе. Контракт проверяется компиляцией и запросами через публичные интерфейсы.
Расширение калькулятора, пп. 4, 6. Для IEcoDSP задано «long double Add(IEcoDSP*, long double, long double);» и аналогичное Subtract.	Дополнительные математические методы не обеспечивали совместимость с существующим IEcoCalculatorX: его IID и целочисленные сигнатуры в инструкции отсутствовали. Клиент этого интерфейса не мог получить его только по наличию Add/Subtract.	Пп. 4, 5: добавлен IEcoCalculatorX с исходным IID и методами Addition/Subtraction. Контракт IEcoDSP сохранен. Проверяются прямой запрос калькулятора через шину, переходы между интерфейсами и общая идентичность экземпляра.
Идентичность, п. 5. «Поддерживай канонический IEcoUnknown.»	Принцип был указан, но не был раскрыт для нескольких интерфейсов одного объекта: общий счетчик, переходы в обе стороны, постоянство набора IID и восстановление адреса владельца для интерфейса со смещением.	П. 5: один канонический Unknown и общий счетчик у IEcoDSP/IEcoCalculatorX. Отдельные дочерние объекты имеют собственную идентичность. Проверяются законы QueryInterface, баланс ссылок и жизнь интерфейса после Release другого интерфейса.
Аллокатор, п. 14. «В компоненте используй только IEcoMemoryAllocator1 для выделения и освобождения памяти.»	Не был определен способ выбора зависимости. Фабрика запрашивала фиксированный CID менеджера; клиент повторно регистрировал стандартный менеджер памяти. Настройка аллокатора среды не учитывалась.	Пп. 3, 9: System подготавливает системные компоненты; фабрика получает System → Bus → MemExt и CID через get_Manager. Проверено, что недоступный выбранный менеджер вызывает отказ без скрытой подмены, а восстановление настройки возвращает работоспособность.

V1: исходная формулировка	Недостаток и следствие	V2: правило и проверка
Изоляция проверок, п. 18. «Интеграционный пример должен собираться и реально выполняться.»	Сам факт запуска примера не определял состав его бинарника. Все клиенты связывались со статическим FFT, а внутренние тесты с tracking allocator выполнялись в основном клиенте. Отдельного доказательства внешней интеграции не было.	П. 18: white-box проверки вынесены в eco_fft_faults. Основной клиент и независимые примеры используют публичный ABI и компонуются с официальным runtime. Анализ символов проверяет отсутствие приватных FFT-функций и вызовов загрузчика в клиентских объектных файлах.
Приемка, п. 21. «Проверь обе формы библиотек»	Работающий статический и динамический расчет не доказывал обнаружение компонента штатной шиной и заменяемость без повторной компоновки. Сценарии отсутствующей, поврежденной и замененной библиотеки не требовались.	Пп. 18, 21: отдельные процессы проверяют отказ без .so, успех после развертывания и замену поставщика под тем же CID при неизменном SHA256 клиента. LD_DEBUG подтверждает разрешение стандартной фабрики. Эти сценарии повторены под ASan/UBSan.

При доработке сохранены уже корректно заданные математические соглашения, числовые типы, сервисный и объектный API, запрет библиотечных вызовов в компоненте и независимые численные эталоны. Изменения усиливают правила интеграции с ACOM и их проверку. Последовательность состояла из анализа V1 и исходного кода, уточнения V2, исправления реализации и приемочных запусков. Сопоставление этих редакций не является контролируемым экспериментом с двумя независимыми генерациями на одной модели; оно документирует проверяемые изменения инструкции и проекта.

15.4. Эталонный пример подключения компонента

Ниже приведен код текущего examples/use_component.c. Это самостоятельный клиент, который использует публичные заголовки и компонуется с официальным Eco runtime. Он не содержит реализации FFT, не регистрирует фабрику и не загружает модуль вручную. Вызовы fft/ifft выполняются через полученный IEcoDSP; все принадлежащие клиенту ссылки освобождаются. Стандартный вывод используется только в клиенте. Рабочий каталог при запуске должен содержать .so с именем CID.

```
#include "IEcoDSP.h"
#include "IEcoSystem1.h"
#include "IEcoInterfaceBus1.h"
#include <stdio.h>

int16_t EcoMain(IEcoUnknown* unknown) {
    IEcoSystem1* system = 0;
    IEcoInterfaceBus1* bus = 0;
    IEcoDSP* dsp = 0;
    EcoComplexD input[4] = {{1, 0}, {2, 0}, {3, 0}, {4, 0}};
    EcoComplexD spectrum[4], restored[4];
    const EcoComplexD expected[4] = {{10, 0}, {-2, 2}, {-2, 0}, {-2, -2}};
    int16_t rc = 1;
    if (!unknown || unknown->pVTbl->QueryInterface(unknown, &GID_IEcoSystem,
(void**)&system)) goto done;
    if (system->pVTbl->QueryInterface(system, &IID_IEcoInterfaceBus1, (void**)&bus))
goto done;
    /* The bus discovers the module and invokes its standard Eco factory. */
    if (bus->pVTbl->QueryComponent(bus, &CID_EcoFFT_DIT, 0, &IID_IEcoDSP, (void**)&dsp))
goto done;
    if (dsp->pVTbl->fft(dsp, 4, input, spectrum)) goto done;
    if (dsp->pVTbl->ifft(dsp, 4, spectrum, restored)) goto done;
    puts("k,fft_real,fft_imag,restored_real,restored_imag");
    for (uint32_t k = 0; k < 4; ++k) {
        double dr = spectrum[k].re - expected[k].re;
        double di = spectrum[k].im - expected[k].im;
        double rr = restored[k].re - input[k].re;
        double ri = restored[k].im;
        if (dr * dr + di * di > 1e-24 || rr * rr + ri * ri > 1e-24) goto done;
        printf("%u,%.12g,%.12g,%.12g,%.12g\n", k, spectrum[k].re, spectrum[k].im,
restored[k].re, restored[k].im);
    }
    rc = 0;
done:
    if (dsp) dsp->pVTbl->Release(dsp);
    if (bus) bus->pVTbl->Release(bus);
    if (system) system->pVTbl->Release(system);
    return rc;
}
```

Если библиотека не развернута, этот клиент завершается ошибкой; после размещения правильной .so получает [10, −2+2i, −2, −2−2i] и восстанавливает [1,2,3,4]. Дополнительный тест scripts/check_acom.py меняет реализацию под тем же CID и проверяет неизменность клиентского бинарника. Пример демонстрирует требуемую границу между потребителем интерфейса и поставщиком реализации.

15.5. Результаты проверки редакции V2

Для текущей поставки Linux x86_64 получены следующие результаты: 134 проверки ACOM-контракта через настоящую шину и 19 сценариев развертывания завершились без ошибок. Они включают правила QueryInterface, идентичность, время жизни, выбор аллокатора, отказ неподдерживаемой агрегации и замену .so без пересборки клиента. Обе

серии повторены с AddressSanitizer и UndefinedBehaviorSanitizer. Протоколы и команды приведены в подразделе 10.5.

Аудит подтвердил неизменность 16 файлов официального SDK, включение исходного depend.h и штатный экспорт фабрики. У двух Release-компонентов отсутствуют неопределенные символы и DT_NEEDED. Математическая часть дополнительно прошла 3936 проверок FFT/API на каждый запуск, 5093 проверки отказов и времени жизни в отдельном тесте и 720 сравнений с FFTW. Числа обозначают разные наборы проверок и не являются процентом покрытия либо количеством независимых генераций модели.

Редакция V2 связывает архитектурное требование с конкретным контрактом и наблюдаемой проверкой. Для следующей генерации компонент принимается только при выполнении всех этих условий: одного наличия vtable, имен QueryInterface/RegisterComponent или правильного спектра недостаточно. Сам промпт не дает безусловной гарантии поведения любой модели; подтверждение соответствия получают повторной сборкой и проверкой созданных артефактов. Агрегация не заявлена и явно отвергается; ее добавление потребует отдельного контракта и собственных проверок.

16. Ограничения и направления развития

EcoFFT предоставляет два ACOM-компонента для FFT/IFFT, единый сервисный контракт и объекты с собственным состоянием. Векторы, матрицы, планы и фильтры создаются через официальный Eco-аллокатор. DIT/DIF имеют отдельные реализации и идентификаторы; произвольные длины обслуживаются через Блюстейна. Компонент доступен как из C-приложения, так и через консольный CSV-интерфейс.

Сервис удобен для разового вычисления и требует меньше постоянной памяти. План сохраняет коэффициенты для повторных расчетов одного размера, но увеличивает постоянную память и затраты на Get/Set и копирование. Измерения показывают фактический баланс этих факторов; наличие кэша не обещает ускорения для каждого размера. Блюстейн требует дополнительных буферов и преобразований по сравнению с radix-2.

Текущая платформа — Linux x86_64; $N \leq 65536$. Каждый объект используется одним потоком либо под внешней синхронизацией. Матрица хранит данные, фильтр выполняет циклическую блочную свертку. Измеренная точность относится к проверенным сигналам и форматам. SIMD, многопоточность, двумерное преобразование, потоковая фильтрация и автоматический выбор плана остаются направлениями дальнейшей разработки. Сравнение с FFTW в режиме FFTW_ESTIMATE приведено в разделе 12; другие режимы планирования и MATLAB не измерялись. Агрегация и управление выгрузкой модулей не предоставляются; время жизни модулей контролирует Eco runtime. Новые интерфейсы добавляются без изменения существующих контрактов.

17. Технические источники

[1] Eco.AI Agentic Coding Meta-Harness. Официальный репозиторий: https://github.com/peerf-eco/eco-harness-releases .

[2] Релиз v0.1.3, SDK и инструменты: https://github.com/peerf-eco/eco-harness-releases/releases/tag/v0.1.3 . Происхождение и SHA256 сохраненных SDK-файлов зафиксированы в vendor/eco/provenance.json.

[3] Cooley J. W., Tukey J. W. An Algorithm for the Machine Calculation of Complex Fourier Series. Mathematics of Computation, 1965, 19(90), 297–301. DOI: https://doi.org/10.1090/S0025-5718-1965-0178586-1 .

[4] Smith J. O. Mathematics of the DFT. Bluestein’s FFT Algorithm. https://www.dsprelated.com/freebooks/mdft/Bluestein_s_FFT_Algorithm.html . Использован математический принцип сведения к свертке; программная реализация самостоятельная.

[5] FFTW. What FFTW Really Computes / Complex One-Dimensional DFTs. https://www.fftw.org/doc/What-FFTW-Really-Computes.html ; https://www.fftw.org/doc/Complex-One_002dDimensional-DFTs.html . Определение преобразований, знак, нормировка и C API библиотеки сравнения; вычислительный компонент от FFTW не зависит.

[6] FFTW. Precision. https://www.fftw.org/doc/Precision.html . Интерфейс fftwl_* и библиотека libfftw3l для long double.

[7] FFTW. Planner Flags. https://www.fftw.org/doc/Planner-Flags.html . Режим FFTW_ESTIMATE, повторное использование wisdom и ограничения планирования.